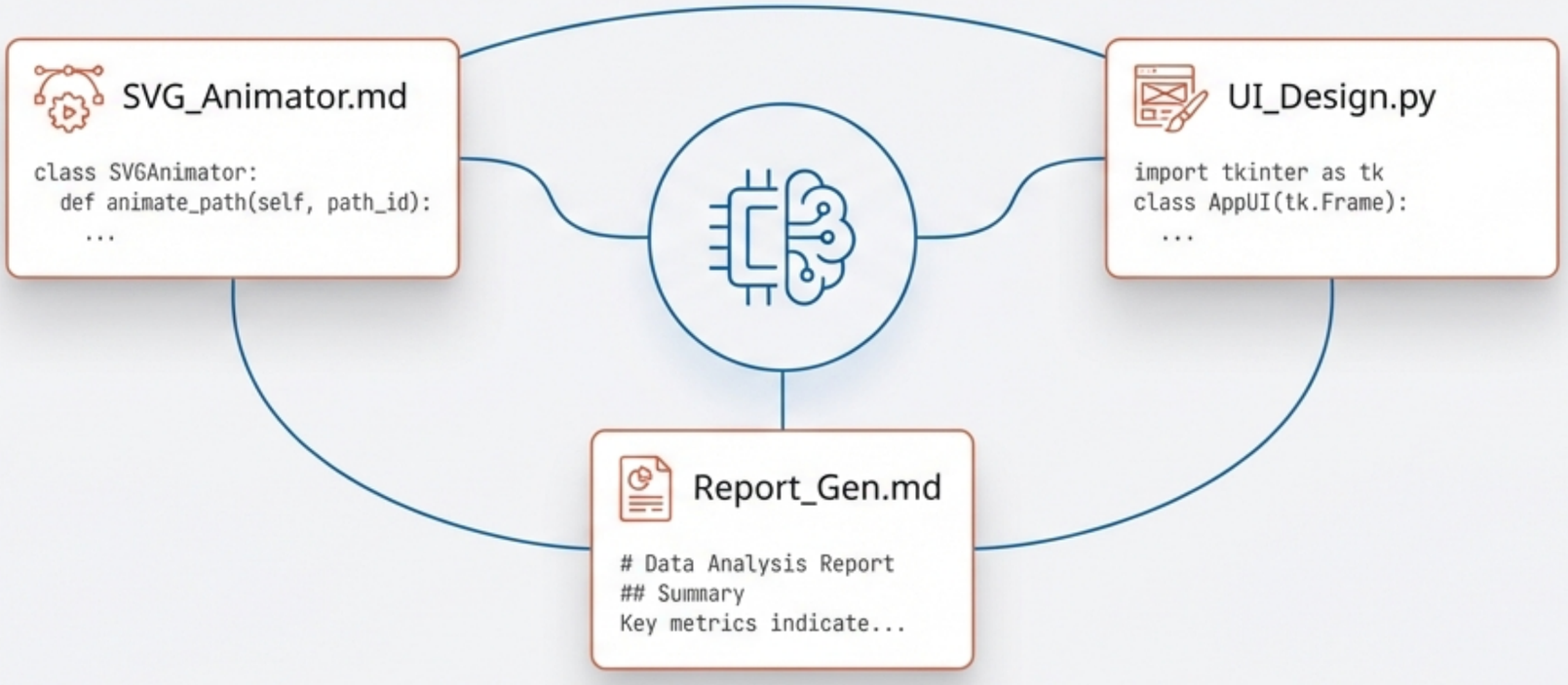
智能体技能：让 AI 从通用聊天变为垂直专家

解决上下文管理与 Token 效率的工程化演进



大多数开发者只使用 AI 进行通用对话。但为了让 AI 掌握复杂工具而不必重复提示，
我们需要一种可扩展的架构。这就是 Agent Skills 的故事。

NotebookLM

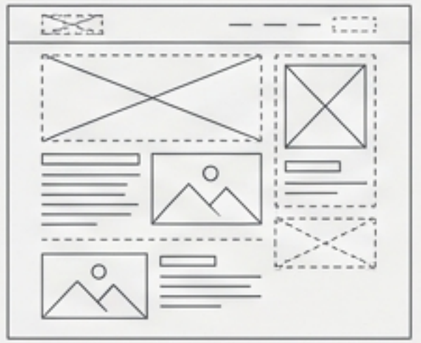
重复劳动的困境：每一次都要从零开始教导

尝试 A

尝试 B

新项目（痛点）

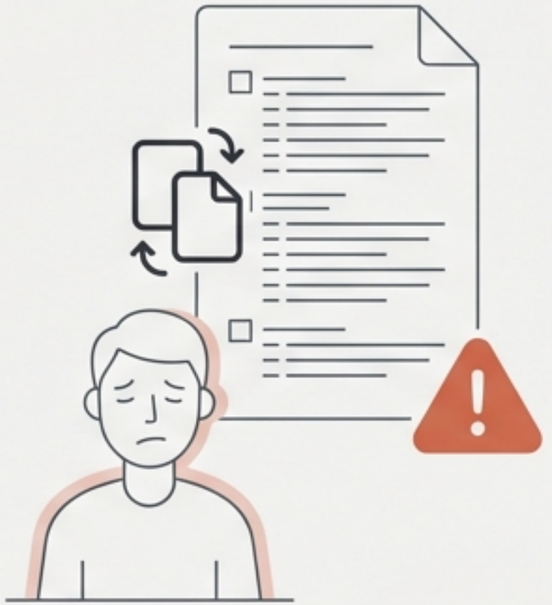
帮我写个博客





无 Emoji，SVG 图标，
专业配色，模块化结构，
响应式设计，字体层级明确
JetBrains Mono...

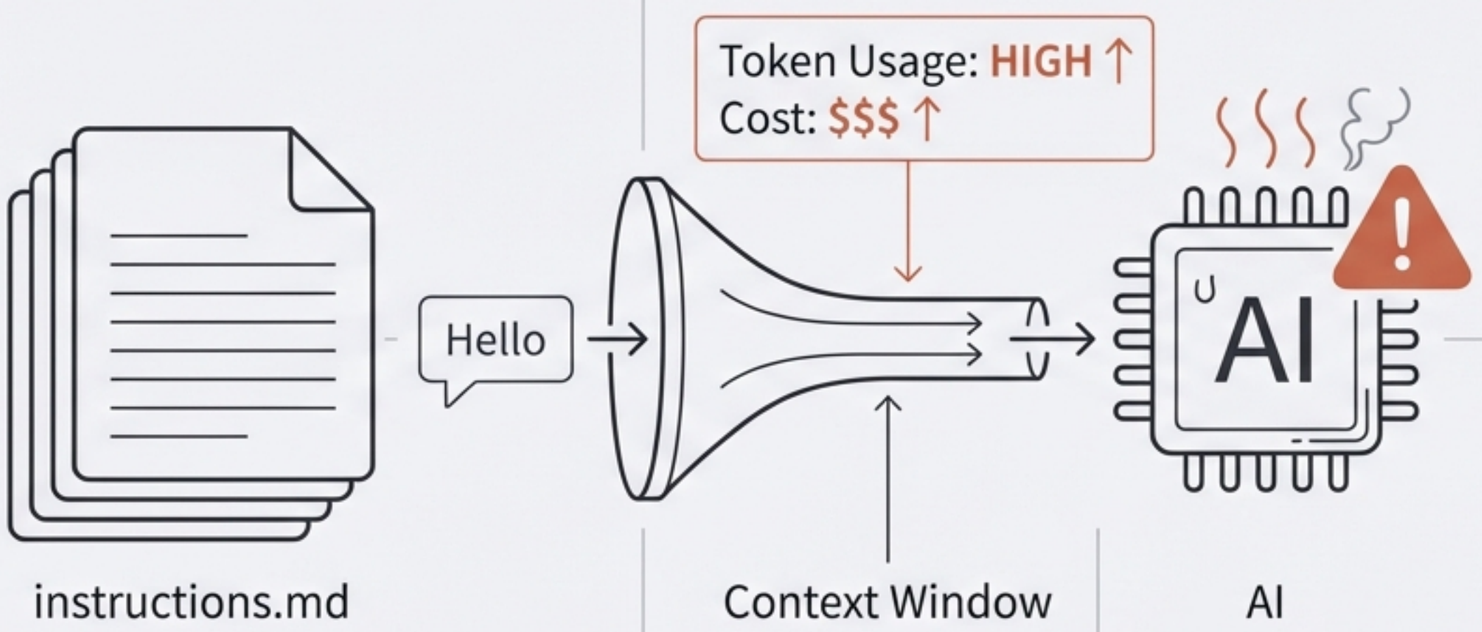




核心洞察：高效的 AI 使用目前受限于上下文管理的各种手动摩擦。我们需要 AI “记住” 标准，而不是每次都重新提醒。

NotebookLM

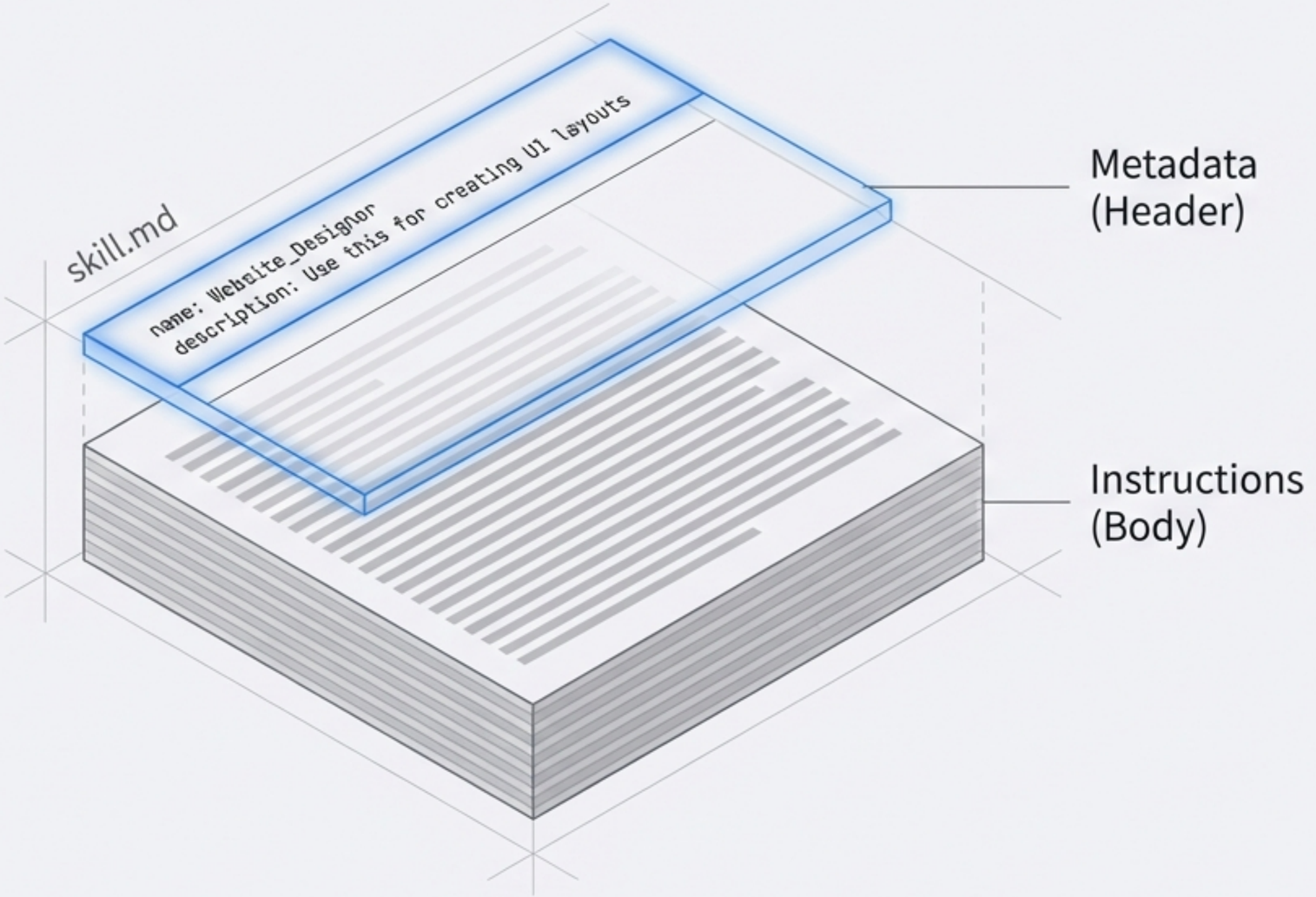
迭代一：外部大脑与 Token 膨胀



- **解决方案**：将所有要求（SVG、无 Emoji、风格指南）写入 Markdown 文件。随请求自动发送。
- **缺陷**：即使是简单的‘你好’，系统也会强制 AI 阅读整本手册。
- **影响**：浪费 Token，响应缓慢，成本高昂。

NotebookLM

迭代二：元数据握手与按需加载



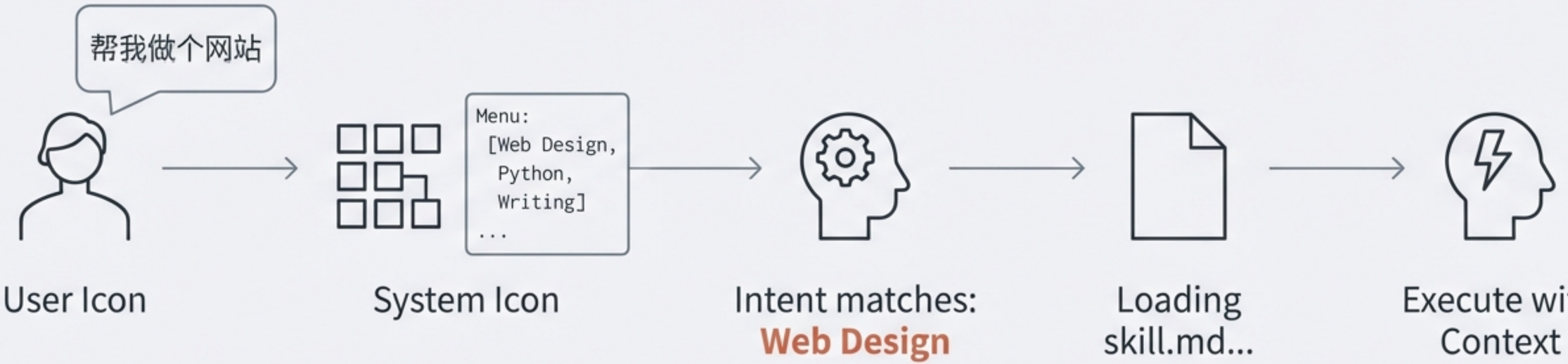
创新点：在 Markdown 文件头部添加名称与描述。

逻辑：系统首先只向 AI 发送菜单（元数据）。只有当用户意图匹配描述时，AI 才会请求完整内容。

核心概念：触发时加载 / 按需加载（Lazy Loading）

NotebookLM

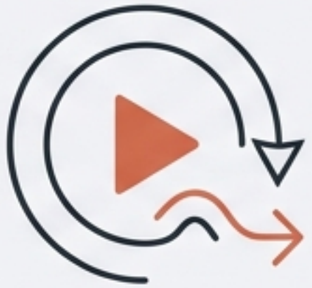
它是如何工作的：Agent Skills 的交互流



这种机制允许 AI 拥有访问无限知识的'潜力'，而无需承担携带所有知识的'动能'成本。

NotebookLM

规模化：构建你的技能库

svg_animator.md	ppt_designer.md	daily_report.md
		
制作网页 SVG 动画	制作美观的 PPT 布局	符合公司风格规范的日报

通过这种文件结构，同一个 AI 模型可以根据激活的文件变身为不同的垂直领域专家。

NotebookLM

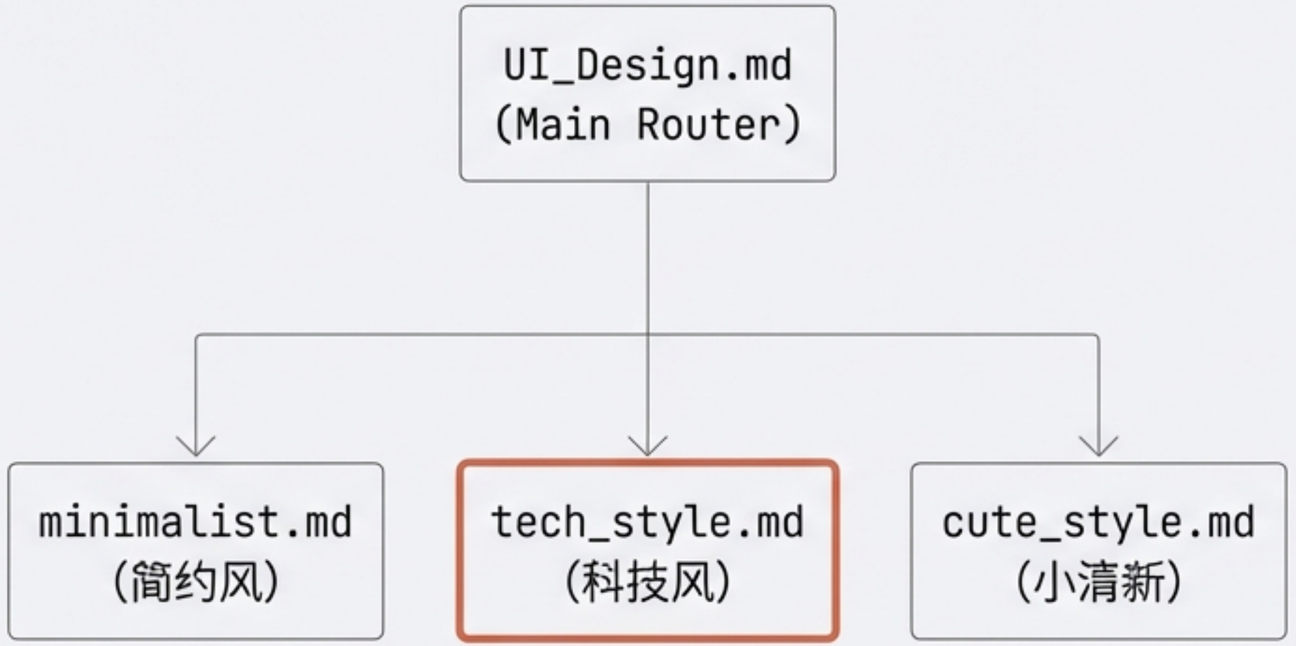
迭代三：复杂度的陷阱与路由模式

问题：单个文件包含所有风格会导致 Token 再次膨胀。

修复：主技能作为**路由器（Router）**。

逻辑：AI 读取用户意图（如“科技风网站”），系统动态加载特定的子文件 `tech_style.md`。

结果：嵌套式按需上下文（Nested Context）。



NotebookLM

迭代四：超越文本—数据与代码执行



需求：纯文本不足以进行精细的 UI 控制（如色值、按钮规格）。

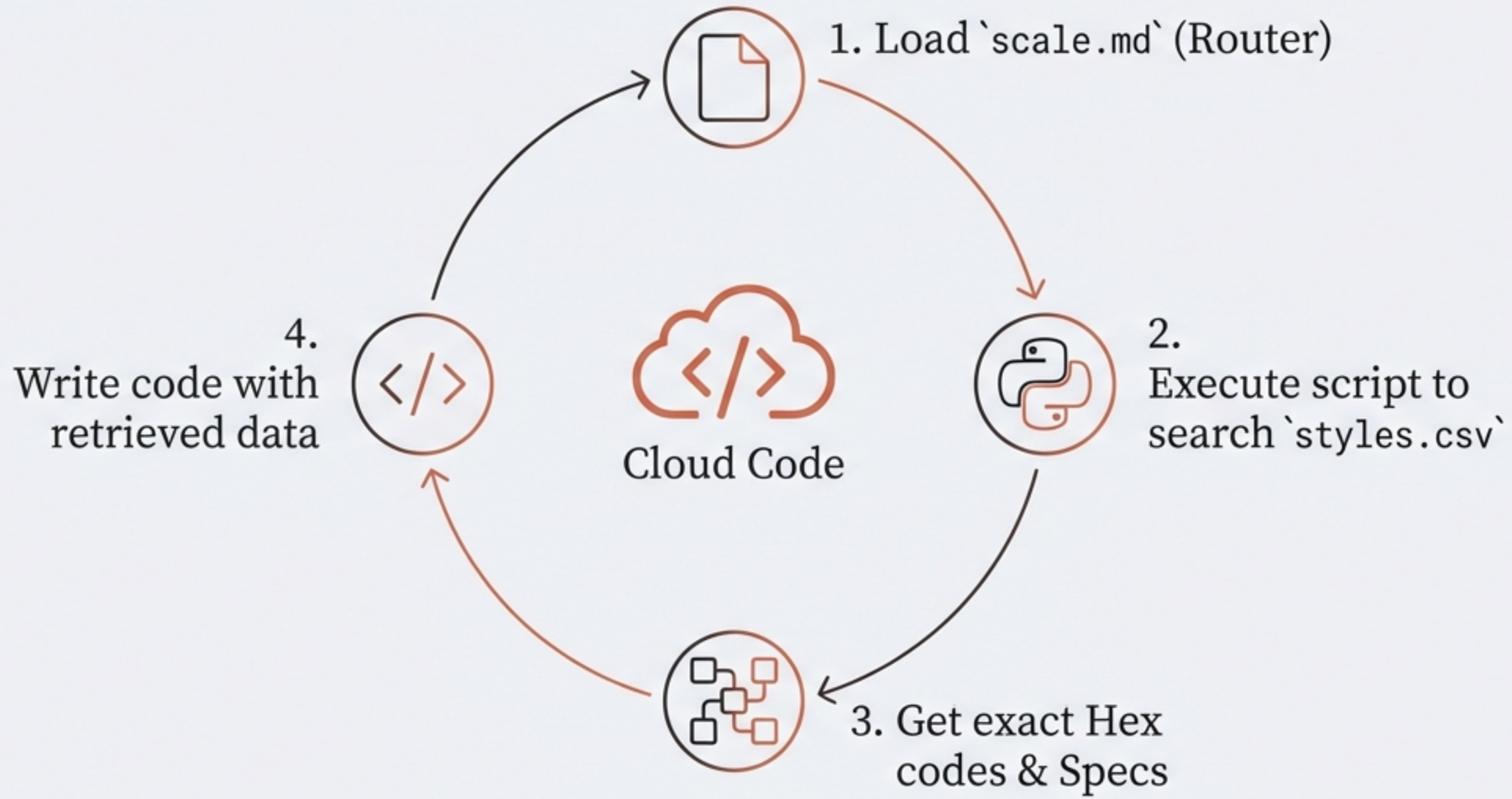
架构：

- 1. *数据*：将 UI 规格存入 `styles.csv`。
- 2. *工具*：提供 `search.py` 脚本。
- 3. *指令*：引导 AI 运行脚本查询表格。

**转变*：智能体从“写手”进化为“操作员”。

NotebookLM

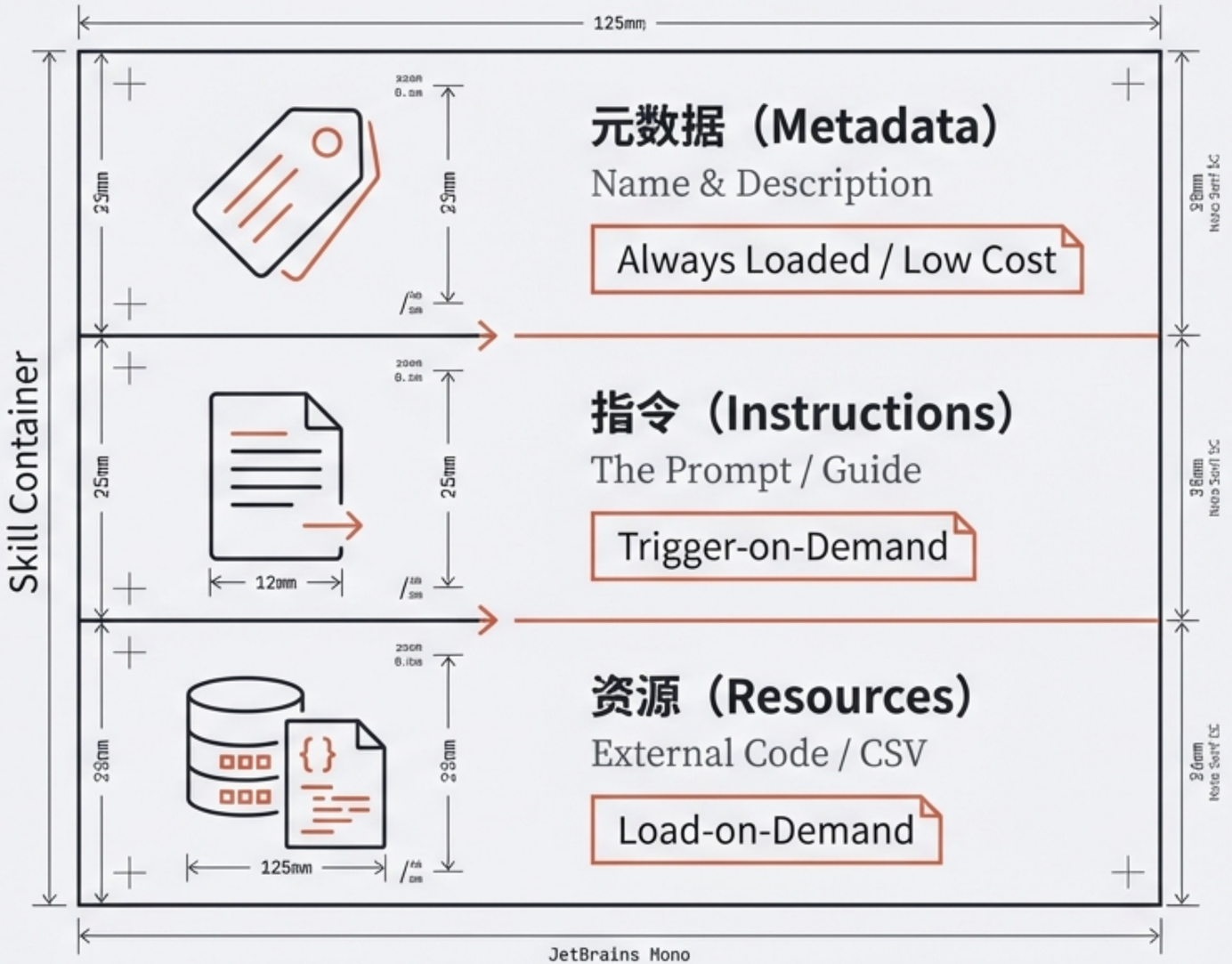
终极形态：全自动化的智能体工作流



这个工作流允许 Cloud Code 交付纯 LLM 无法准确记忆的、数据驱动的精确结果。

NotebookLM

剖析：Agent Skill 的三大核心组件



NotebookLM

真实世界的验证



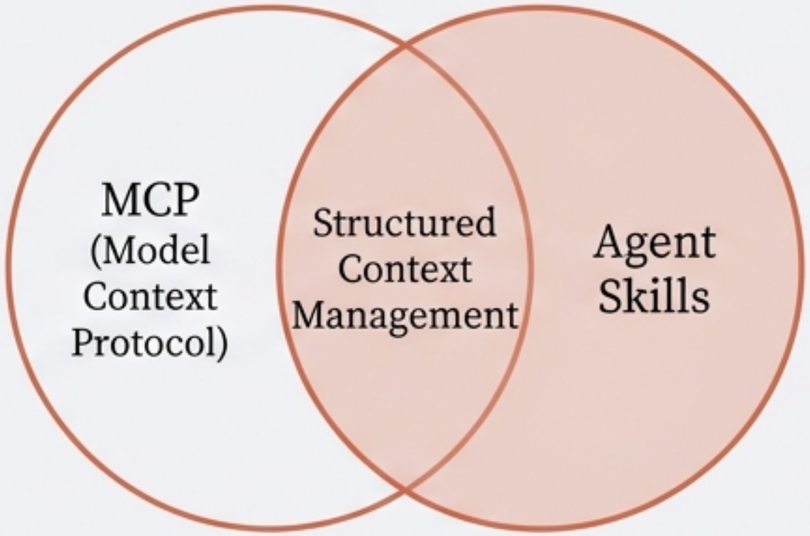
这里展示的案例是真实存在的。

影响力*: 这种模式正在驱动下一代编程助手 （Cloud Code）。它实现了更高质量、更低延迟和更好的可维护性。

NotebookLM

本质：工程化的提示词管理

Agent Skills + Context ≈ Prompt Engineering



去掉术语，这就是高级的 **提示词工程（Prompt Engineering）**。无论是叫 Skills 还是 Protocols，目标都是一致的：结构化、高效的上下文管理。

NotebookLM

为什么你需要构建 Agent Skills



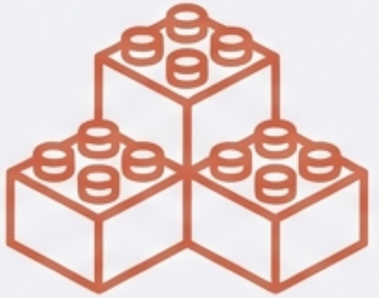
效率（Efficiency）

节省 Token，降低延迟。



质量（Quality）

强制严格遵守复杂规则（UI，风格，格式）。

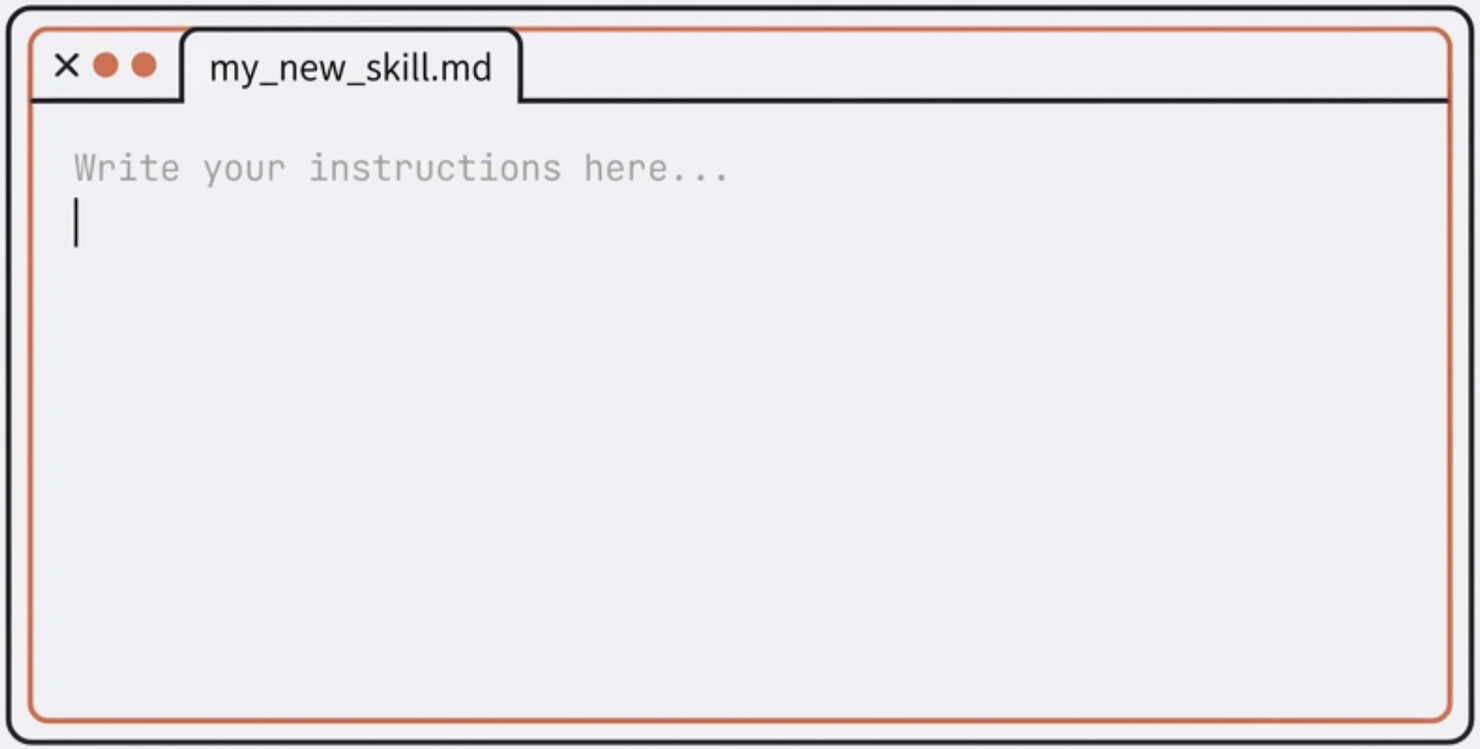


模块化（Modularity）

以文件形式管理能力，而非巨型文本块。

NotebookLM

下一个 Skill，由你定义



从简单的 Markdown 指南开始，进化到代码执行。将你的 AI 从通用助手转变为垂直合作伙伴。

Noto Serif SC Regular, Slate Grey (#4A4A4A)

Call to Action: 欢迎在评论区分享你的 Skill。

NotebookLM